VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

ADARSH K P

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by ADARSH KP (1BM24CS015), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.
The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

SEEMA PATIL
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1a.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	
1b.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: ● Rate- Monotonic ● Earliest-deadline First ● Proportional scheduling	
4.	Write a C program to simulate producer-consumer problem using semaphores Write a C program to simulate the concept of Dining Philosophers problem	
5.	Write a C program to simulate Bankers algorithm and deadlock avoidance.	
6.	Write a C program to simulate Memory Allocation	
7.	Write a C program to simulate Page Replacement	

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program1a

Question: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

- FCFS
- SJF (pre-emptive & Non-preemptive)

Soln: =>FCFS

```
#include <stdio.h>
#include <stdlib.h>

struct process {
   int id, at, bt, ct, tat, wt, rt;
};

int main() {
   int n;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct process p[n];
   for (int i = 0; i < n; i++) {
      p[i].id = i + 1;
      printf("Enter Arrival Time for P%d: ", p[i].id);
      scanf("%d", &p[i].at);
      printf("Enter Burst Time for P%d: ", p[i].id);
      scanf("%d", &p[i].bt);
   }
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (p[j].at > p[j + 1].at) {
            struct process temp = p[j];
            p[j] = p[j + 1];
            p[j + 1] = temp;
         }
      }
   }
   int timepassed = 0, sumtat = 0, sumwt = 0, sumrt = 0;
   for (int i = 0; i < n; i++) {
      if (timepassed < p[i].at) {
         timepassed = p[i].at;
      }
      p[i].ct = timepassed + p[i].bt;
      p[i].tat = p[i].ct - p[i].at;
      p[i].wt = p[i].tat - p[i].bt;

      timepassed += p[i].bt;

      sumtat += p[i].tat;
      sumwt += p[i].wt;

   }
   printf("\nID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
```

```
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id, p[i].at, p[i].bt, p[i].ct,
               p[i].tat, p[i].wt);
    }
    printf("\nAverage TAT: %.2f\n", (float)sumtat / n);
    printf("Average WT: %.2f\n", (float)sumwt / n);

    return 0;
}
```

```
[adarsh@Adarshs-MacBook-Air FCFS % gcc fcfs.c
[adarsh@Adarshs-MacBook-Air FCFS % ./a.out

enter id, at and bt respectively
1 0 7

enter id, at and bt respectively
2 8 3

enter id, at and bt respectively
3 3 4

enter id, at and bt respectively
4 5 6

P_id     AT      BT      CT      TAT     WT

1        0       7       7       7       0

3        3       4       11      8       4

4        5       6       17      12      6

2        8       3       20      12      9
adarsh@Adarshs-MacBook-Air FCFS %
```

SOLN: =>SJF(Preemptive)

```
#include<stdio.h>

struct process {
   int id, at, bt, ct, tat, wt, rt, rem;
};

int main() {
   int n;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct process p[n];
   for (int i = 0; i < n; i++) {
      p[i].id = i + 1;
      printf("Enter Arrival Time for P%d: ", p[i].id);
      scanf("%d", &p[i].at);
      printf("Enter Burst Time for P%d: ", p[i].id);
      scanf("%d", &p[i].bt);
      p[i].rem = p[i].bt;
      p[i].rt = -1;
   }

   int completed = 0, time = 0, sumtat = 0, sumwt = 0, sumrt = 0;
while (completed != n) {
   int idx = -1, minrem = 1e9;
   for (int i = 0; i < n; i++) {
     if (p[i].at <= time && p[i].rem > 0) {
          if (p[i].rem < minrem ||(p[i].rem==minrem&&p[i].at<p[idx].at)) {
             minrem = p[i].rem;
             idx = i;
          }
      }
   }
   if (idx == -1) {
      time++;
      continue;
   }
   if (p[idx].rt == -1) {
      p[idx].rt = time - p[idx].at;
      sumrt += p[idx].rt;
   }
   p[idx].rem--;
   time++;
   if (p[idx].rem == 0) {
      completed++;
      p[idx].ct = time;
      p[idx].tat = p[idx].ct - p[idx].at;
      p[idx].wt = p[idx].tat - p[idx].bt;
      sumtat += p[idx].tat;
      sumwt += p[idx].wt;
   }
 }
```

```
    printf("\nID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id, p[i].at, p[i].bt, p[i].ct,
               p[i].tat, p[i].wt);
    }

    printf("\nAverage TAT: %.2f\n", (float)sumtat / n);
    printf("Average WT: %.2f\n", (float)sumwt / n);
    return 0;
}
```

```
[adarsh@Adarshs-MacBook-Air SJF_Pre % gcc sjf_preemptive.c
[adarsh@Adarshs-MacBook-Air SJF_Pre % ./a.out
 Enter Arrival & Burst for P1: 0 8
 Enter Arrival & Burst for P2: 1 4
 Enter Arrival & Burst for P3: 2 9
 Enter Arrival & Burst for P4: 3 5

 ID      AT      BT      CT      TAT     WT
 -----------------------------------------
 P1      0       8       17      17      9
 P2      1       4       5       4       0
 P3      2       9       26      24      15
 P4      3       5       10      7       2
```

SOLN: => SJF Non preemptive

```
#include<stdio.h>
struct process{
   int id,at,bt,ct,tat,wt,rt;
};

int main() {
   int n;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct process p[n];
   for (int i = 0; i < n; i++){
      p[i].id = i + 1;
      printf("Enter Arrival Time for P%d: ", p[i].id);
      scanf("%d", &p[i].at);
      printf("Enter Burst Time for P%d: ", p[i].id);
      scanf("%d", &p[i].bt);
   }

   int completed=0,time=0,sumtat=0,sumwt=0;
   int iscompleted[n];
   for(int i=0;i<n;i++) iscompleted[i]=0;

   while(completed!=n){
      int index=-1;
      int minbt=1e9;
      for(int i=0;i<n;i++){
         if(p[i].at<=time && !iscompleted[i]){
            if(p[i].bt<minbt){
               minbt=p[i].bt;
               index=i;
            }
         }
      }
      if(index!=-1){
         p[index].ct=time+p[index].bt;
         p[index].tat=p[index].ct-p[index].at;
         p[index].wt=p[index].tat-p[index].bt;
         p[index].rt=p[index].wt;
         sumtat+=p[index].tat;
         sumwt+=p[index].wt;
         time=p[index].ct;
         iscompleted[index]=1;
         completed++;
      } else {
         time++;
      }
   }

   printf("\nID\tAT\tBT\tCT\tTAT\tWT\n");
   for(int i=0;i<n;i++){
      printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
   }
```

```
    printf("\nAverage TAT: %.2f", (float)sumtat/n);
    printf("\nAverage WT: %.2f", (float)sumwt/n);
}
```

```
[adarsh@Adarshs-MacBook-Air DS_practice % gcc sjf_nonpreemptive.c
[adarsh@Adarshs-MacBook-Air DS_practice % ./a.out

enter id, at and bt
2 0 3

enter id, at and bt
3 0 4

enter id, at and bt
4 0 6

enter id, at and bt
1 0 7

P_id     AT      BT      CT      TAT     WT

2        0       3       3       3       0

3        0       4       7       7       3

4        0       6       13      13      7

1        0       7       20      20      13
adarsh@Adarshs-MacBook-Air DS_practice %
```

PROGRAM 1b

Question : Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin (Experiment with different quantum sizes for RR algorithm)

SOLN: Priority Pre-emptive

```
#include <stdio.h>

struct process {
   int pid, at, bt, p, rt, ct, wt, tat;
};

int main() {
   int n, time = 0, completed = 0;
   printf("Enter the number of processes: ");
   scanf("%d", &n);
   struct process p[n];

   for (int i = 0; i < n; i++) {
      p[i].pid = i + 1;
      printf("Enter the arrival time, burst time, priority: ");
      scanf("%d%d%d", &p[i].at, &p[i].bt, &p[i].p);
      p[i].rt = p[i].bt;
   }

   while (completed < n) {
      int idx = -1, highest = 9999;
      for (int i = 0; i < n; i++) {
         if (p[i].at <= time && p[i].rt > 0) {
            if (p[i].p < highest) {
               highest = p[i].p;
               idx = i;
            }
         }
      }
      if (idx != -1) {
         p[idx].rt--;
         time++;
         if (p[idx].rt == 0) {
            completed++;
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
         }
      } else {
         time++;
      }
   }

   float totalwt = 0, totaltat = 0;
   printf("\nPid\tAt\tBt\tP\tCt\tTat\tWt\n");
```

```
    for (int i = 0; i < n; i++) {
        totalwt += p[i].wt;
        totaltat += p[i].tat;
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].p,
               p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage waiting time : %.2f", totalwt / n);
    printf("\nAverage turnaround time : %.2f\n", totaltat / n);

    return 0;
}
```

```
Enter number of processes: 7
Enter AT, BT and Priority for P1: 0 8 3
Enter AT, BT and Priority for P2: 1 2 4
Enter AT, BT and Priority for P3: 3 4 4
Enter AT, BT and Priority for P4: 4 1 5
Enter AT, BT and Priority for P5: 5 6 2
Enter AT, BT and Priority for P6: 6 5 6
Enter AT, BT and Priority for P7: 10 1 1

PID     AT      BT      Pri     CT      WT      TAT
1       0       8       3       15      7       15
2       1       2       4       17      14      16
3       3       4       4       21      14      18
4       4       1       5       22      17      18
5       5       6       2       12      1       7
6       6       5       6       27      16      21
7       10      1       1       11      0       1

Average CT: 17.86
Average WT: 9.86
Average TAT: 13.71
```

SOLN : Non premetive

```
#include <stdio.h>

struct process {
   int id, at, bt, pr, ct, tat, wt, d;
};

int main() {
   int n, t = 0, c = 0, h;
   float tw = 0, tt = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   struct process p[n];

   for (int i = 0; i < n; i++) {
      p[i].id = i + 1;
      printf("Process %d\n", p[i].id);
      printf("Arrival time: ");
      scanf("%d", &p[i].at);
      printf("Burst time: ");
      scanf("%d", &p[i].bt);
      printf("Priority: ");
      scanf("%d", &p[i].pr);
      p[i].d = 0;
   }

   while (c != n) {
      h = -1;
      for (int i = 0; i < n; i++) {
         if (p[i].at <= t && p[i].d == 0) {
            if (h == -1 || p[i].pr < p[h].pr) {
               h = i;
            }
         }
      }

      if (h == -1) {
         t++;
      } else {
         t += p[h].bt;
         p[h].ct = t;
         p[h].tat = p[h].ct - p[h].at;
         p[h].wt = p[h].tat - p[h].bt;
         tw += p[h].wt;
         tt += p[h].tat;
         p[h].d = 1;
         c++;
      }
   }

   printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
   for (int i = 0; i < n; i++) {
```

```
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id, p[i].at, p[i].bt, p[i].pr,
               p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time: %.2f", tt / n);
    printf("\nAverage Waiting Time: %.2f\n", tw / n);

    return 0;
}
```

```
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0 3 5
Enter AT, BT and Priority for P2: 2 2 3
Enter AT, BT and Priority for P3: 3 5 2
Enter AT, BT and Priority for P4: 4 4 4
Enter AT, BT and Priority for P5: 6 1 1

PID     AT      BT      Pri     CT      WT      TAT
1       0       3       5       3       0       3
2       2       2       3       11      7       9
3       3       5       2       8       0       5
4       4       4       4       15      7       11
5       6       1       1       9       2       3
```

SOLN : Round Robin

```
#include <stdio.h>

struct Process {
   int pid, at, bt, ct, tat, wt, rt;
};

int main() {
   int n, tq;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   struct Process p[n];
   int rem_bt[n];
   int readyQ[100];
   int front = 0, rear = 0;

   printf("Enter arrival times:\n");
   for(int i=0; i<n; i++) {
      scanf("%d", &p[i].at);
      p[i].pid = i+1;
   }

   printf("Enter burst times:\n");
   for(int i=0; i<n; i++) {
      scanf("%d", &p[i].bt);
      rem_bt[i] = p[i].bt;
      p[i].rt = p[i].bt;
      p[i].ct = p[i].tat = p[i].wt = 0;
   }

   int time = 0, completed = 0;
   int visited[n];
   for(int i=0; i<n; i++) visited[i] = 0;


   for(int i=0; i<n; i++) {
      if(p[i].at == 0) {
         readyQ[rear++] = i;
         visited[i] = 1;
      }
   }

   while(completed < n) {
      if(front == rear) {
         time++;
         for(int i=0; i<n; i++) {
            if(!visited[i] && p[i].at <= time) {
               readyQ[rear++] = i;
               visited[i] = 1;
            }
```

```
            }
            continue;
        }

        int idx = readyQ[front++];
        if(rem_bt[idx] > tq) {
            time += tq;
            rem_bt[idx] -= tq;
            p[idx].rt = rem_bt[idx];
        } else {
            time += rem_bt[idx];
            rem_bt[idx] = 0;
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            p[idx].rt = 0;
            completed++;
        }


        for(int i=0; i<n; i++) {
            if(!visited[i] && p[i].at <= time) {
                readyQ[rear++] = i;
                visited[i] = 1;
            }
        }

        if(rem_bt[idx] > 0) {
            readyQ[rear++] = idx;
        }
    }

    printf("\nRRS scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].ct,
               p[i].tat, p[i].wt);
    }


    float avg_TAT = 0, avg_WT = 0;
    for(int i=0; i<n; i++) {
        avg_TAT += p[i].tat;
        avg_WT += p[i].wt;
    }

    printf("\nAverage turnaround time: %.2f ms\n", avg_TAT/n);
    printf("Average waiting time: %.2f ms\n", avg_WT/n);

    return 0;
}
```

```
Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
5 3 1 2 3

Round Robin Scheduling:
PID     AT      BT      CT      TAT     WT      RT
1       0       5       13      13      8       0
2       1       3       12      11      8       0
3       2       1       5       3       2       0
4       3       2       9       6       4       0
5       4       3       14      10      7       0

Average turnaround time: 8.60 ms
Average waiting time: 5.80 ms

1BM24CS015
```

PROGRAM 2

Question : Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

SOLN :

```
#include <stdio.h>

#define MAX 10

typedef struct {
int pid, at, bt, rt, ct, tat, wt, queue;
} Process;

int main() {
Process p[MAX];
int n, i, time = 0, completed = 0;
int tq = 2;

printf("Enter number of processes: ");
scanf("%d", &n);

for (i = 0; i < n; i++) {
printf("\nP%d\n", i + 1);
p[i].pid = i + 1;

printf("Arrival Time: ");
scanf("%d", &p[i].at);

printf("Burst Time: ");
scanf("%d", &p[i].bt);

printf("Queue (1-System, 2-User): ");
scanf("%d", &p[i].queue);

p[i].rt = p[i].bt;
}

printf("\nGantt Chart:\n");

while (completed < n) {

int executed = 0;

for (i = 0; i < n; i++) {
if (p[i].queue == 1 && p[i].rt > 0 && p[i].at <= time) {

executed = 1;

if (p[i].rt > tq) {
printf("P%d ", p[i].pid);
time += tq;
```

```
p[i].rt -= tq;
} else {
printf("P%d ", p[i].pid);
time += p[i].rt;
p[i].rt = 0;
p[i].ct = time;
completed++;
}
}
}

if (!executed) {
for (i = 0; i < n; i++) {
if (p[i].queue == 2 && p[i].rt > 0 && p[i].at <= time) {

executed = 1;

printf("P%d ", p[i].pid);
p[i].rt--;
time++;

if (p[i].rt == 0) {
p[i].ct = time;
completed++;
}

break;
}
}
}

if (!executed) {
time++;
}
}

float total_tat = 0, total_wt = 0;

printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++) {
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;

total_tat += p[i].tat;
total_wt += p[i].wt;

printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
     p[i].pid, p[i].at, p[i].bt,
     p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage TAT = %.2f", total_tat / n);
printf("\nAverage WT = %.2f\n", total_wt / n);
```

```
return 0;
}
```

```
[adarsh@Adarshs-MacBook-Air C % ./a.out
 Enter number of processes: 4
 Enter Time Quantum for Q1: 2
 P1 [AT, BT, Q(1=Sys, 2=User)]: 0 4 1
 P2 [AT, BT, Q(1=Sys, 2=User)]: 0 3 1
 P3 [AT, BT, Q(1=Sys, 2=User)]: 0 8 2
 P4 [AT, BT, Q(1=Sys, 2=User)]: 10 5 1

 ID      Q       AT      BT      CT      TAT     WT
 1       1       0       4       6       6       2
 2       1       0       3       7       7       4
 3       2       0       8       15      15      7
 4       1       10      5       20      10      5

 Avg TAT: 9.50 | Avg WT: 4.50
```

PROGRAM 3

QUESTION : Write a C program to simulate Real-Time CPU Scheduling algorithms:
Rate- Monotonic
Earliest-deadline First
Proportional scheduling

SOLN : Rate Monotonic Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
struct Process {
   int pid;
   int burst;
   int period;
   int remaining;
};

int gcd(int a, int b) {
   if (b == 0) return a;
   return gcd(b, a % b);
}
int lcm(int a, int b) {
   return (a * b) / gcd(a, b);
}

int main() {
   int n;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   struct Process p[n];
   printf("Enter the CPU burst times:\n");
   for (int i = 0; i < n; i++) {
      scanf("%d", &p[i].burst);
      p[i].pid = i + 1;
   }

   printf("Enter the time periods:\n");
   int hyperPeriod = 1;
   for (int i = 0; i < n; i++) {
      scanf("%d", &p[i].period);
      hyperPeriod = lcm(hyperPeriod, p[i].period);
   }

   printf("\nLCM=%d\n", hyperPeriod);
```

```
    printf("\nRate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\n", p[i].pid, p[i].burst, p[i].period);
    }


    double U = 0.0;
    for (int i = 0; i < n; i++) {
        U += (double)p[i].burst / p[i].period;
    }
    double bound = n * (pow(2.0, 1.0/n) - 1);
    printf("\n%.6f <= %.6f => %s\n", U, bound, (U <= bound ? "true" : "false"));
    printf("Scheduling occurs for %d ms\n\n", hyperPeriod);


    for (int t = 0; t < hyperPeriod; t++) {
        int chosen = -1;

        for (int i = 0; i < n; i++) {
            if (t % p[i].period == 0) {
                p[i].remaining = p[i].burst;
            }
        }

        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                chosen = i;
                break;
            }
        }
        if (chosen != -1) {
            printf("%dms onwards: Process %d running\n", t, p[chosen].pid);
            p[chosen].remaining--;
        } else {
            printf("%dms onwards: CPU is idle\n", t);
        }
    }

    return 0;
}
```

```
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100

LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
```

SOLN : Earliest Deadline First

```
#include <stdio.h>
#include <limits.h>

#define MAX_TASKS 10
typedef struct {
    int pid;
    int burst;
    int deadline;
    int period;
} Task;

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;
    Task tasks[MAX_TASKS];

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nTask %d:\n", i + 1);
        tasks[i].pid = i + 1;

        printf("Enter Burst Time: ");
        scanf("%d", &tasks[i].burst);

        printf("Enter Deadline: ");
        scanf("%d", &tasks[i].deadline);

        printf("Enter Period: ");
        scanf("%d", &tasks[i].period);
    }

    int SIMULATION_TIME = tasks[0].period;
```

```
for (int i = 1; i < n; i++) {
   SIMULATION_TIME = lcm(SIMULATION_TIME, tasks[i].period);
}

int remaining[MAX_TASKS] = {0};
int next_release[MAX_TASKS] = {0};
int abs_deadline[MAX_TASKS] = {0};
int active[MAX_TASKS] = {0};

printf("\nScheduling occurs for %d ms (LCM of periods)\n\n", SIMULATION_TIME);

for (int t = 0; t < SIMULATION_TIME; t++) {

   // Release jobs
   for (int i = 0; i < n; i++) {
      if (t == next_release[i]) {
         remaining[i] = tasks[i].burst;
         abs_deadline[i] = t + tasks[i].deadline;
         next_release[i] += tasks[i].period;
         active[i] = 1;
      }
   }

   int chosen = -1;
   int earliest = INT_MAX;

   for (int i = 0; i < n; i++) {
      if (active[i] && remaining[i] > 0) {
         if (abs_deadline[i] < earliest) {
            earliest = abs_deadline[i];
            chosen = i;
         }
      }
   }

   if (chosen != -1) {
      printf("%dms : Task %d is running.\n", t, tasks[chosen].pid);
      remaining[chosen]--;

      if (remaining[chosen] == 0) {
         active[chosen] = 0;
      }
   } else {
      printf("%dms : CPU is idle.\n", t);
   }
}
```

```
    return 0;
}
```

```
Enter PID, Burst, Deadline, and Period for process 2: 2 2 4 5
Enter PID, Burst, Deadline, and Period for process 3: 3 2 8 10

PID     Burst   Deadline        Period
1       3       7               20
2       2       4               5
3       2       8               10
Scheduling occurs for 20 ms

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms: CPU is idle.
10ms : Task 2 is running.
11ms : Task 2 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms: CPU is idle.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms: CPU is idle.
18ms: CPU is idle.
19ms: CPU is idle.
```

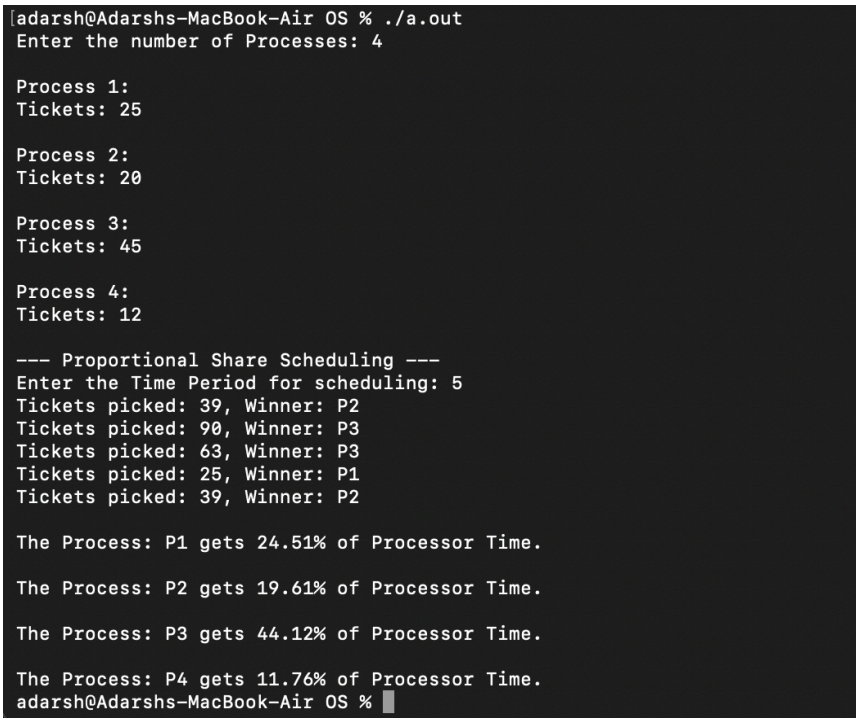
SOLN : Proportional Share Scheduling

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));
```

} return 0; }

Output:



PROGRAM 4 :

QUESTION :
4a. Write a C program to simulate producer-consumer problem using semaphores
4b. Write a C program to simulate the concept of Dining Philosophers problem

SOLN : Bounded Buffer

```
#include <stdio.h>
#include <stdlib.h>
#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

int empty = BUFFER_SIZE;
int full = 0;

void produce() {
   if (empty == 0) {
      printf("\nBuffer is full! Cannot produce.\n");
      return;
   }

   int item;
   printf("Enter item to produce: ");
   scanf("%d", &item);

   buffer[in] = item;
   printf("Produced: %d\n", item, in);

   in = (in + 1) % BUFFER_SIZE;
   empty--;
   full++;
}

void consume() {
   if (full == 0) {
      printf("Buffer is empty! Cannot consume.\n");
      return;
   }

   int item = buffer[out];
   printf("Consumed: %d\n", item, out);

   out = (out + 1) % BUFFER_SIZE;
```

```
    full--;
    empty++;
}

int main() {
    int choice;
    printf("Buffer Size: %d\n", BUFFER_SIZE);

    while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\n");
        printf("Current State: [Full slots: %d | Empty slots: %d]\n", full, empty);
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1) break;

        switch (choice) {
            case 1:
                produce();
                break;
            case 2:
                consume();
                break;
            case 3:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

```
adarsh@Adarshs-MacBook-Air OS % ./a.out
Buffer Size: 5

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 1
Enter item to produce: 12
Produced: 12

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 4
Invalid choice!

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 3
Exiting...
adarsh@Adarshs-MacBook-Air OS %
```

SOLN : Dining Philosopher

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num)
{
int id = *(int*)num;
int left = id; // left fork index
int right = (id + 1) % N; // right fork index
while (1)
{
printf("Philosopher %d is thinking.\n", id);
sleep(1); // Thinking
// To avoid deadlock, philosophers with even id pick left then right,
// odd id pick right then left.
if (id % 2 == 0)
{

pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); }
else
{
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
}
printf("Philosopher %d is eating.\n", id);
sleep(2); // Put down forks
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

int main() {
int i;
int ids[N]; // Initialize forks (mutexes)
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
```

```
ids[i] = i;
}
for (i = 0; i < N; i++){
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}

for (i = 0; i < N; i++){
pthread_join(philosophers[i], NULL); }
for (i = 0; i < N; i++){
pthread_mutex_destroy(&forks[i]);
}
return 0;
}
```

```
[adarsh@Adarshs-MacBook-Air OS % ./a.out
 Philosopher 0 is thinking.
 Philosopher 1 is thinking.
 Philosopher 4 is thinking.
 Philosopher 3 is thinking.
 Philosopher 2 is thinking.
 Philosopher 0 picked up left fork 0.
 Philosopher 0 picked up right fork 1.
 Philosopher 4 picked up left fork 4.
 Philosopher 0 is eating.
 Philosopher 2 picked up left fork 2.
 Philosopher 2 picked up right fork 3.
 Philosopher 2 is eating.
 Philosopher 2 put down forks 2 and 3.
 Philosopher 2 is thinking.
 Philosopher 1 picked up right fork 2.
 Philosopher 0 put down forks 0 and 1.
 Philosopher 0 is thinking.
 Philosopher 4 picked up right fork 0.
 Philosopher 4 is eating.
 Philosopher 1 picked up left fork 1.
 Philosopher 1 is eating.
 Philosopher 4 put down forks 4 and 0.
 Philosopher 4 is thinking.
 Philosopher 3 picked up right fork 4.
 Philosopher 3 picked up left fork 3.
 Philosopher 3 is eating.
 Philosopher 0 picked up left fork 0.
 Philosopher 1 put down forks 1 and 2.
 Philosopher 1 is thinking.
 Philosopher 0 picked up right fork 1.
 Philosopher 0 is eating.
 Philosopher 2 picked up left fork 2.
```

PROGRAM 5

QUESTION : Write a C program to simulate Bankers algorithm and deadlock avoidance

SOLN : Banker algorithm

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main() {
   int n, m;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resource types: ");
   scanf("%d", &m);

   int allocation[MAX_PROCESSES][MAX_RESOURCES];
   int max[MAX_PROCESSES][MAX_RESOURCES];
   int need[MAX_PROCESSES][MAX_RESOURCES];
   int available[MAX_RESOURCES];

   printf("\nEnter Allocation Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &allocation[i][j]);
      }
   }

   printf("\nEnter Max Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &max[i][j]);
      }
   }

   printf("\nEnter Available Resources:\n");
   for (int j = 0; j < m; j++) {
      scanf("%d", &available[j]);
   }

   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
```

```
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

bool finish[MAX_PROCESSES] = {false};
int safeSequence[MAX_PROCESSES];

int work[MAX_RESOURCES];

// Work = Available
for (int j = 0; j < m; j++) {
    work[j] = available[j];
}

int count = 0;

while (count < n) {
    bool found = false;

    for (int i = 0; i < n; i++) {

        if (finish[i] == false) {

            bool canExecute = true;

            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canExecute = false;
                    break;
                }
            }

            if (canExecute) {

                for (int j = 0; j < m; j++) {
                    work[j] += allocation[i][j];
                }

                safeSequence[count] = i;
                count++;

                finish[i] = true;
                found = true;
            }
        }
    }
```

```
            if (!found) {
                break;
            }
        }

        if (count == n) {
            printf("\nSystem is in SAFE state.\n");
            printf("Safe Sequence: ");

            for (int i = 0; i < n; i++) {
                printf("P%d", safeSequence[i]);

                if (i != n - 1) {
                    printf(" -> ");
                }
            }

            printf("\n");
        }
        else {
            printf("\nSystem is NOT in safe state.\n");
        }

        return 0;
}
```

```
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

SOLN : Deadlock avoidance algorithm

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main() {
   int n, m;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resource types: ");
   scanf("%d", &m);

   int allocation[MAX_PROCESSES][MAX_RESOURCES];
   int request[MAX_PROCESSES][MAX_RESOURCES];
   int available[MAX_RESOURCES];

   printf("\nEnter Allocation Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &allocation[i][j]);
      }
   }

   printf("\nEnter Request Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         scanf("%d", &request[i][j]);
      }
   }

   printf("\nEnter Available Resources:\n");
   for (int j = 0; j < m; j++) {
      scanf("%d", &available[j]);
   }

   bool finish[MAX_PROCESSES];
   int work[MAX_RESOURCES];
   int safeSequence[MAX_PROCESSES];

   for (int j = 0; j < m; j++) {
      work[j] = available[j];
```

```
}

for (int i = 0; i < n; i++) {
   bool zeroAllocation = true;

   for (int j = 0; j < m; j++) {
      if (allocation[i][j] != 0) {
         zeroAllocation = false;
         break;
      }
   }

   finish[i] = zeroAllocation;
}

int count = 0;

while (1) {
   bool found = false;

   for (int i = 0; i < n; i++) {

      if (finish[i] == false) {

         bool canExecute = true;

         for (int j = 0; j < m; j++) {
            if (request[i][j] > work[j]) {
               canExecute = false;
               break;
            }
         }

         if (canExecute) {

            for (int j = 0; j < m; j++) {
               work[j] += allocation[i][j];
            }

            finish[i] = true;
            safeSequence[count] = i;
            count++;

            found = true;
         }
      }
   }
```

```
            if (!found) {
                break;
            }
        }

        bool deadlock = false;

        for (int i = 0; i < n; i++) {
            if (finish[i] == false) {
                deadlock = true;
                break;
            }
        }

        if (!deadlock) {
            printf("\nNo Deadlock detected.\n");

            printf("Safe Sequence: ");
            for (int i = 0; i < count; i++) {
                printf("P%d", safeSequence[i]);

                if (i != count - 1) {
                    printf(" -> ");
                }
            }

            printf("\n");
        }
        else {
            printf("\nDeadlock detected.\n");

            printf("Processes involved in deadlock: ");

            for (int i = 0; i < n; i++) {
                if (finish[i] == false) {
                    printf("P%d ", i);
                }
            }

            printf("\n");
        }

        return 0;
    }
```

```
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
000
0 0

No Deadlock detected.
Safe Sequence: P0 -> P2 -> P3 -> P4 -> P1
```

PROGRAM 6 :

QUESTION : Write a C program to simulate Memory Allocation (First Fit, Best Fit, Worst Fit)

SOLN : Memory Allocation

```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] = 0;
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;

        for (int j = 0; j < m; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] = 0;
        }
    }

    printf("\nBest Fit Allocation:\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;

        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] = 0;
        }
```

```
    }

    printf("\nWorst Fit Allocation:\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blocks[m], firstBlocks[m], bestBlocks[m], worstBlocks[m];

    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blocks[i]);
        firstBlocks[i] = blocks[i];
        bestBlocks[i] = blocks[i];
        worstBlocks[i] = blocks[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    firstFit(firstBlocks, m, processSize, n);
    bestFit(bestBlocks, m, processSize, n);
    worstFit(worstBlocks, m, processSize, n);

    return 0;
}
```

```
PS C:\Users\BMSCECSE-SH\Desktop\Abhishek K S 1BM24CS010 OS lab Memory> & "C:\Users\BMSCECSE-SH\Desktop\Abhishek K S 1BM24CS010 OS lab Memory\MemoryAllocation.exe"
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
412
112
426

First Fit Allocation:
Process No.     Process Size    Block No.
1               212             2
2               412             5
3               112             3
4               426             Not Allocated

Best Fit Allocation:
Process No.     Process Size    Block No.
1               212             4
2               412             2
3               112             3
4               426             5

Worst Fit Allocation:
Process No.     Process Size    Block No.
1               212             5
2               412             2
3               112             4
4               426             Not Allocated
PS C:\Users\BMSCECSE-SH\Desktop\Abhishek K S 1BM24CS010 OS lab Memory>
```

PROGRAM 7 :

QUESTION : Write a C program to simulate Page Replacement

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PAGES 100

void printFrames(int frames[], int num_frames) {
   printf("[");
   int count = 0;
   for (int i = 0; i < num_frames; i++) {
      if (frames[i] != -1) {
         printf("%d", frames[i]);
         count++;
      }
   }
   printf("]\n");
}


void fifoPageReplacement(int pages[], int num_pages, int num_frames) {
   printf("\n--- FIFO Page Replacement ---\n");

   int frames[num_frames];
   for (int i = 0; i < num_frames; i++) frames[i] = -1;

   int page_faults = 0;
   int next = 0;

   for (int i = 0; i < num_pages; i++) {
      int page = pages[i];
      bool is_present = false;


      for (int j = 0; j < num_frames; j++) {
         if (frames[j] == page) {
            is_present = true;
            break;
         }
      }

      printf("Page %d -> ", page);

      if (!is_present) {
```

```
            frames[next] = page;
            next = (next + 1) % num_frames;
            page_faults++;
        }

        printFrames(frames, num_frames);
    }
    printf("Total Page Faults (FIFO): %d\n", page_faults);
}


void lruPageReplacement(int pages[], int num_pages, int num_frames) {
    printf("\n--- LRU Page Replacement ---\n");

    int frames[num_frames];
    int last_used[num_frames];
    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
        last_used[i] = -1;
    }

    int page_faults = 0;

    for (int i = 0; i < num_pages; i++) {
        int page = pages[i];
        bool is_present = false;
        int present_index = -1;


        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                is_present = true;
                present_index = j;
                break;
            }
        }

        printf("Page %d -> ", page);

        if (is_present) {

            last_used[present_index] = i;
        } else {

            page_faults++;
```

```
            int empty_index = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    empty_index = j;
                    break;
                }
            }

            if (empty_index != -1) {

                frames[empty_index] = page;
                last_used[empty_index] = i;
            } else {

                int lru_index = 0;
                int min_time = last_used[0];
                for (int j = 1; j < num_frames; j++) {
                    if (last_used[j] < min_time) {
                        min_time = last_used[j];
                        lru_index = j;
                    }
                }
                frames[lru_index] = page;
                last_used[lru_index] = i;
            }
        }

        printFrames(frames, num_frames);
    }
    printf("Total Page Faults (LRU): %d\n", page_faults);
}


void optimalPageReplacement(int pages[], int num_pages, int num_frames) {
    printf("\n--- Optimal Page Replacement ---\n");

    int frames[num_frames];
    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    int page_faults = 0;

    for (int i = 0; i < num_pages; i++) {
        int page = pages[i];
        bool is_present = false;

        for (int j = 0; j < num_frames; j++) {
```

```
    if (frames[j] == page) {
        is_present = true;
        break;
    }
}

printf("Page %d -> ", page);

if (!is_present) {
    page_faults++;

    int empty_index = -1;
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == -1) {
            empty_index = j;
            break;
        }
    }

    if (empty_index != -1) {
        frames[empty_index] = page;
    } else {

        int replace_index = -1;
        int farthest_use = -1;

        for (int j = 0; j < num_frames; j++) {
            int next_use = -1;

            for (int k = i + 1; k < num_pages; k++) {
                if (pages[k] == frames[j]) {
                    next_use = k;
                    break;
                }
            }


            if (next_use == -1) {
                replace_index = j;
                break;
            }


            if (next_use > farthest_use) {
                farthest_use = next_use;
                replace_index = j;
            }
```

```
            }

            frames[replace_index] = page;
        }
    }

    printFrames(frames, num_frames);
  }
  printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main() {
  int num_pages, num_frames;
  int pages[MAX_PAGES];


  printf("Enter number of pages: ");
  if (scanf("%d", &num_pages) != 1 || num_pages <= 0 || num_pages > MAX_PAGES) {
    printf("Invalid number of pages.\n");
    return 1;
  }

  printf("Enter page reference string:\n");
  for (int i = 0; i < num_pages; i++) {
    scanf("%d", &pages[i]);
  }

  printf("Enter number of frames: ");
  if (scanf("%d", &num_frames) != 1 || num_frames <= 0) {
    printf("Invalid number of frames.\n");
    return 1;
  }


  fifoPageReplacement(pages, num_pages, num_frames);
  lruPageReplacement(pages, num_pages, num_frames);
  optimalPageReplacement(pages, num_pages, num_frames);

  return 0;
}
```

```
PS C:\Users\BMSCECSE-L4\Desktop\OS lab7 Page replacement Abhishek ks 1BM24CS010> & "C:\Users\BMSCECSE-L4\Desktop\OS lab7 Page replacement Abhishek ks 1BM24CS010\PageRep.exe"
Enter number of pages: 21
Enter page reference string:
5
0
1
0
2
3
0
2
4
3
3
2
0
2
1
2
7
0
1
1
0
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [231]
Page 0 -> [230]
Page 2 -> [230]
Page 4 -> [430]
Page 3 -> [430]
Page 3 -> [430]
Page 2 -> [420]
Page 0 -> [420]
Page 2 -> [420]
Page 1 -> [421]
Page 2 -> [421]
Page 7 -> [721]
Page 0 -> [701]
Page 1 -> [701]
Page 1 -> [701]
Page 0 -> [701]
Total Page Faults (FIFO): 11

--- LRU Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
```

```
--- LRU Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 2 -> [203]
Page 4 -> [204]
Page 3 -> [234]
Page 3 -> [234]
Page 2 -> [234]
Page 0 -> [230]
Page 2 -> [230]
Page 1 -> [210]
Page 2 -> [210]
Page 7 -> [217]
Page 0 -> [207]
Page 1 -> [107]
Page 1 -> [107]
Page 0 -> [107]
Total Page Faults (LRU): 12

--- Optimal Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 2 -> [203]
Page 4 -> [243]
Page 3 -> [243]
Page 3 -> [243]
Page 2 -> [243]
Page 0 -> [203]
Page 2 -> [203]
Page 1 -> [201]
Page 2 -> [201]
Page 7 -> [701]
Page 0 -> [701]
Page 1 -> [701]
Page 1 -> [701]
Page 0 -> [701]
Total Page Faults (Optimal): 9
PS C:\Users\BMSCECSE-L4\Desktop\OS lab7 Page replacement Abhishek ks 1BM24CS010>
```